MODUL 05 · BAB II · ANALISIS RUNNING TIME

Pertumbuhan Fungsi Running Time

Seberapa cepat waktu eksekusi membengkak ketika data bertambah, dan mengapa memilih algoritma lebih penting daripada membeli prosesor lebih cepat.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menyebutkan delapan bentuk umum fungsi running time dan mengurutkannya dari yang paling lambat tumbuh.
- Menghitung rasio pertumbuhan $f(n_2)/f(n_1)$ untuk berbagai bentuk fungsi, termasuk yang memuat logaritma.
- Menggunakan sifat logaritma ($\lg x^a = a \lg x$, perubahan basis) dalam perhitungan rasio.
- Membandingkan pengaruh pemilihan algoritma dengan pengaruh kecepatan prosesor melalui perhitungan.

PRASYARAT

- Modul 03: running time $T(n)$ dan orde pertumbuhan.
- Modul 04: contoh running time linier dan kuadrat.

BAB / MATERI ASAL

Materi 3. PERTUMBUHAN FUNGSI RUNNING TIME
PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Kompleksitas Algoritma)

POSISI DALAM URUTAN BELAJAR

Menjembatani perhitungan running time (Modul 03–04) dengan notasi asimtotik formal (Modul 06–07).

DAFTAR ISI

1. Ketika Data Bertambah Banyak 1
2. Bentuk-Bentuk Umum Fungsi Running Time 2
3. Rasio Pertumbuhan 2
4. Studi Kasus: Prosesor Cepat atau Algoritma Tepat? 4
5. Dari Rasio ke Notasi Asimtotik 6
6. Kesimpulan 7
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 9
- Referensi 9

PENDAHULUAN

1. Ketika Data Bertambah Banyak

Di Modul 04 kita menemukan bahwa running time Insertion Sort bisa berbentuk linier ($an + b$) atau kuadrat ($an^2 + bn + c$). Untuk data kecil, perbedaannya hampir tidak terasa. Namun aplikasi nyata sering mengolah ribuan sampai jutaan data, dan di sinilah perbedaan bentuk fungsi menjadi sangat menentukan.

Pertanyaan yang dijawab modul ini adalah: **jika ukuran data naik, berapa kali lipat waktu eksekusinya naik?** Jawabannya bergantung sepenuhnya pada bentuk fungsi running time. Kita akan mengukurnya

dengan **rasio pertumbuhan**, membandingkan delapan bentuk fungsi yang paling umum, lalu memakai hasilnya untuk menjawab pertanyaan praktis dari bahan kuliah: lebih efisien mana, memakai prosesor yang lebih cepat atau memilih algoritma yang tepat?

PEMBAHASAN INTI

2. Bentuk-Bentuk Umum Fungsi Running Time

Running time suatu algoritma membentuk fungsi $f(n)$. Bentuk-bentuk yang paling sering muncul, diurutkan dari yang paling lambat tumbuh ke yang paling cepat tumbuh, adalah:

No	Fungsi	Nama	Contoh algoritma
1	c	Konstanta	Mengakses elemen array dengan indeks tertentu
2	$c_2 \lg n$	Logaritmik	Binary Search (Modul 02)
3	$c_3 n$	Linier	Menelusuri seluruh array; Insertion Sort kasus terbaik
4	$c_4 n \lg n$	$n \lg n$ (linearitmik)	Merge Sort (Modul 08)
5	$c_5 n^2$	Kuadrat	Insertion Sort kasus terburuk (Modul 04)
6	$c_6 n^3$	Kubik	Matrix-Chain-Order (Modul 11)
7	$c_7 2^n$	Eksponensial	Mencoba semua himpunan bagian dari n objek
8	$c_8 n!$	Faktorial	Mencoba semua permutasi n objek (*brute force*)

Tabel 1. Delapan bentuk umum fungsi running time.

Fungsi 1–6 tergolong **polinomial** (atau dibatasi oleh polinomial). Fungsi 7 dan 8 tumbuh jauh lebih cepat daripada polinomial mana pun, sehingga algoritma dengan running time seperti itu hanya praktis untuk n yang sangat kecil.

2.1 Pengingat logaritma

Fungsi 2 dan 4 memuat logaritma. Beberapa notasi dan sifat yang dipakai:

- $\lg x$ adalah logaritma basis 2: $\lg x = {}^2\log x$. Contoh: $\lg 8 = 3$, $\lg 1024 = 10$.
- $\log x$ (tanpa keterangan) adalah logaritma basis 10: $\log 1000 = 3$.
- Perubahan basis: ${}^b\log x = \dfrac{\log x}{\log b}$. Jadi $\lg x = \dfrac{\log x}{\log 2}$.
- Sifat pangkat: $\lg x^a = a \lg x$. Contoh: $\lg 25 = \lg 5^2 = 2 \lg 5$.

> **CATATAN**
>
> Karena $\lg x = \log x / \log 2$, logaritma dengan basis berbeda hanya berselisih **faktor konstan** ($1/\log 2 \approx 3{,}32$). Dalam analisis orde pertumbuhan, faktor konstan diabaikan, sehingga basis logaritma tidak memengaruhi bentuk pertumbuhan. Algoritma dengan running time $\lg n$ dan $\log n$ sama-sama disebut logaritmik.

PEMBAHASAN INTI

3. Rasio Pertumbuhan

DEFINISI

Jika ukuran masukan naik dari n_1 menjadi n_2, **rasio pertumbuhan** fungsi running time f adalah

$$\text{Rasio} = \frac{f(n_2)}{f(n_1)}.$$

Rasio ini menyatakan berapa kali lipat waktu eksekusi bertambah. Konstanta c_i selalu saling menghapus, sehingga rasio hanya bergantung pada **bentuk** fungsi.

Contoh dari bahan kuliah: rasio kenaikan dari $n = 5$ menjadi $n = 25$ untuk setiap bentuk fungsi.

Fungsi	Perhitungan rasio $f(25)/f(5)$	Rasio
c	$\frac{c}{c}$	1
$c_2 \lg n$	$\frac{c_2 \lg 25}{c_2 \lg 5} = \frac{\lg 5^2}{\lg 5} = \frac{2 \lg 5}{\lg 5}$	2
$c_3 n$	$\frac{c_3 \cdot 25}{c_3 \cdot 5}$	5
$c_4 n \lg n$	$\frac{25 \lg 25}{5 \lg 5} = \frac{25 \cdot 2 \lg 5}{5 \lg 5} = \frac{50}{5}$	10
$c_5 n^2$	$\frac{25^2}{5^2} = 5^2$	25
$c_6 n^3$	$\frac{25^3}{5^3} = 5^3$	125
$c_7 2^n$	$\frac{2^{25}}{2^5} = 2^{25-5} = 2^{20}$	1.048.576
$c_8 n!$	$\frac{25!}{5!} = 25 \cdot 24 \cdot 23 \cdots 6$	$\approx 1{,}29 \times 10^{23}$

Tabel 2. Rasio pertumbuhan dari n = 5 menjadi n = 25.

Ukuran data hanya naik 5 kali, tetapi akibatnya sangat berbeda: fungsi konstan tidak berubah sama sekali, fungsi logaritmik hanya naik 2 kali, fungsi kuadrat naik 25 kali, sedangkan fungsi eksponensial naik lebih dari sejuta kali.

MEMBACA MAKNA RASIO

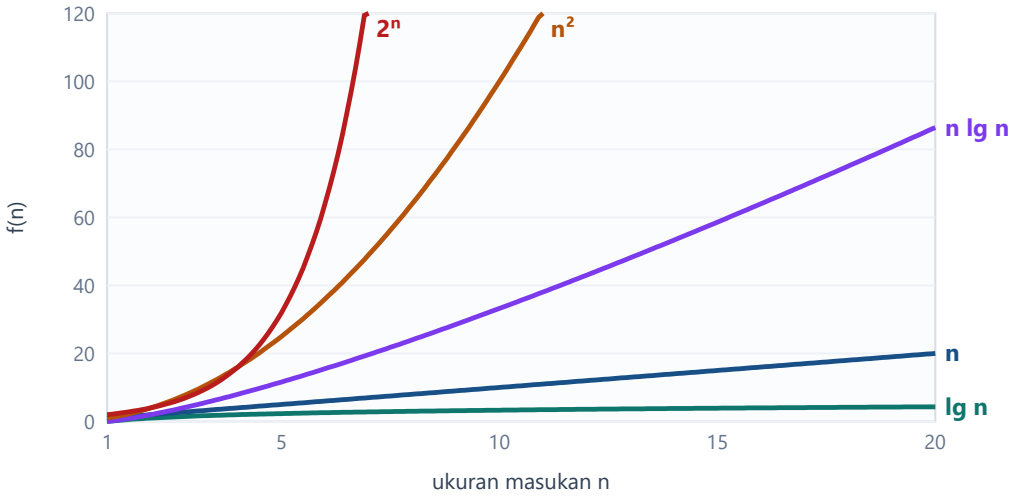
Bahan kuliah menyatakan: "untuk nilai rasio semakin kecil, berarti running time dari algoritma tersebut semakin cepat." Lebih tepatnya, rasio kecil berarti running time **bertambah lambat** ketika data bertambah, sehingga algoritma tersebut **lebih efisien untuk data besar** (*scalable*). Rasio tidak membandingkan kecepatan dua algoritma pada ukuran data yang sama, karena konstanta c_i sudah saling menghapus.

3.1 Nilai fungsi untuk n yang semakin besar

n	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
10	3,3	10	33	100	1.000	1.024
100	6,6	100	664	10.000	10^6	$1{,}27 \times 10^{30}$
1.000	10,0	1.000	9.966	10^6	10^9	$1{,}07 \times 10^{301}$
10^6	19,9	10^6	$1{,}99 \times 10^7$	10^{12}	10^{18}	sangat besar

Tabel 3. Nilai beberapa fungsi pertumbuhan (dibulatkan).

Agar angka ini terasa nyata, misalkan sebuah komputer menjalankan 10^9 langkah dasar per detik (asumsi ilustratif). Untuk $n = 10^6$, algoritma $n \lg n$ selesai dalam sekitar **0,02 detik**, algoritma n^2 membutuhkan sekitar **1.000 detik (±17 menit)**, dan algoritma n^3 membutuhkan 10^9 detik, lebih dari **31 tahun**. Algoritma 2^n bahkan tidak dapat diselesaikan untuk $n = 100$: dibutuhkan sekitar $1{,}27 \times 10^{21}$ detik, jauh melebihi umur alam semesta.



Gambar 1. Grafik lg n, n, n lg n, n^2, dan 2^n untuk n = 1 sampai 20 (sumbu tegak dipotong di 120). Kurva 2^n dan n^2 melesat ke atas jauh lebih awal daripada yang lain.

CONTOH DAN PENERAPAN

4. Studi Kasus: Prosesor Cepat atau Algoritma Tepat?

Soal dari bahan kuliah:

SOAL

Algoritma A dengan $f(n) = c_4\, n \lg n$ dan algoritma B dengan $f(n) = c_5\, n^2$ membutuhkan waktu yang sama, yaitu T detik, untuk mengurutkan data sebanyak $n = 4$.

a. Berapa waktu yang dibutuhkan algoritma A dan B untuk data sebanyak $n = 1024$ jika keduanya dijalankan di komputer yang sama?

b. Jika algoritma B dijalankan di komputer dengan prosesor lebih cepat, mana yang lebih efisien: memakai prosesor yang lebih cepat atau memilih algoritma yang tepat?

4.1 Penyelesaian (a)

Algoritma A, $f(n) = c_4\, n \lg n$:

$$\text{Rasio}_A = \frac{c_4 \cdot 1024 \lg 1024}{c_4 \cdot 4 \lg 4} = \frac{1024 \lg 4^5}{4 \lg 4} = \frac{1024 \cdot 5 \lg 4}{4 \lg 4} = 1280.$$

(Di sini dipakai $1024 = 4^5$ dan $\lg 4^5 = 5 \lg 4$. Cara lain: $\lg 1024 = 10$ dan $\lg 4 = 2$, sehingga $\frac{1024 \cdot 10}{4 \cdot 2} = 1280$.)

Algoritma B, $f(n) = c_5\, n^2$:

$$\text{Rasio}_B = \frac{c_5 \cdot 1024^2}{c_5 \cdot 4^2} = \frac{(4^5)^2}{4^2} = \left(\frac{1024}{4}\right)^2 = 256^2 = 65536.$$

Karena untuk $n = 4$ keduanya membutuhkan T detik, untuk $n = 1024$ algoritma A membutuhkan $1280\,T$ **detik** dan algoritma B membutuhkan $65536\,T$ **detik**. Algoritma B kira-kira 51 kali lebih lambat.

4.2 Penyelesaian (b)

Misalkan B dijalankan di komputer yang prosesornya **20 kali** lebih cepat. Waktunya menjadi

$$\frac{65536\,T}{20} = 3276{,}8\,T \text{ detik},$$

masih lebih lambat daripada algoritma A di komputer biasa ($1280\,T$ detik). **Kesimpulan: lebih efisien memilih algoritma yang tepat.**

> **CATATAN KOREKSI TERHADAP SUMBER**
>
> Soal di bahan kuliah menyebut prosesor "2 kali lebih cepat", tetapi penyelesaiannya memakai "20 kali lebih cepat" ($65536T/20$). Keduanya memberikan kesimpulan yang sama: dengan prosesor 2 kali lebih cepat, B membutuhkan $65536T/2 = 32768\,T$ detik; dengan prosesor 20 kali lebih cepat, $3276{,}8\,T$ detik. Keduanya tetap lebih lambat dari A ($1280\,T$). Agar B menyamai A untuk $n = 1024$, prosesornya harus sekitar $65536/1280 = 51{,}2$ kali lebih cepat, dan untuk n yang lebih besar lagi kebutuhan percepatannya terus naik.

4.3 Mengapa prosesor cepat tidak cukup

Percepatan prosesor hanya membagi waktu dengan **faktor konstan**. Sebaliknya, selisih antara n^2 dan $n \lg n$ adalah faktor $\frac{n}{\lg n}$ yang **terus membesar** seiring n. Untuk n yang cukup besar, faktor yang terus membesar pasti mengalahkan faktor konstan sebesar apa pun.

4.4 Titik potong: kapan algoritma "lebih lambat tumbuh" mulai menang?

Algoritma yang pertumbuhannya lebih lambat belum tentu lebih cepat untuk data kecil, karena konstantanya bisa lebih besar. Misalkan Insertion Sort membutuhkan $2n^2$ langkah dan Merge Sort $50\, n \lg n$ langkah (konstanta ilustratif). Program berikut mencari n terkecil saat Merge Sort mulai lebih cepat.

Python: rasio pertumbuhan dan titik potong

```python
import math
lg = math.log2

# rasio n = 5 -> 25 untuk setiap bentuk fungsi
bentuk = {"c": lambda n: 1, "lg n": lg, "n": lambda n: n,
          "n lg n": lambda n: n * lg(n), "n^2": lambda n: n**2,
          "n^3": lambda n: n**3, "2^n": lambda n: 2**n}
for nama, f in bentuk.items():
    print(f"{nama:7s} {f(25) / f(5):,.0f}")

# studi kasus n = 4 -> 1024
print(1024 * lg(1024) / (4 * lg(4)), 1024**2 / 4**2)

# titik potong 2n^2 vs 50 n lg n
n = 2
while 2 * n * n <= 50 * n * lg(n):
    n += 1
print("Merge Sort mulai lebih cepat pada n =", n)
```

Keluaran program

```
c       1
lg n    2
n       5
n lg n  10
n^2     25
n^3     125
2^n     1,048,576
1280.0 65536.0
Merge Sort mulai lebih cepat pada n = 190
```

Untuk $n < 190$, Insertion Sort (dengan konstanta kecil) lebih cepat; untuk $n \geq 190$, Merge Sort selalu lebih cepat, dan keunggulannya terus bertambah. Inilah alasan praktis mengapa beberapa pustaka pengurutan memakai Insertion Sort untuk potongan data yang kecil.

ANALISIS DAN HUBUNGAN KONSEP

5. Dari Rasio ke Notasi Asimtotik

Yang penting adalah perilaku untuk n besar. Studi kasus di atas menunjukkan dua hal sekaligus. Untuk n kecil, konstanta bisa membuat algoritma yang "buruk" terlihat lebih cepat. Untuk n besar, bentuk fungsi selalu menang. Analisis algoritma karena itu memusatkan perhatian pada perilaku fungsi ketika n menuju tak hingga, yang disebut perilaku **asimtotik**.

Konstanta dan suku orde rendah diabaikan. Rasio pertumbuhan tidak dipengaruhi konstanta c_i. Modul 03 juga menunjukkan bahwa suku berpangkat rendah semakin tidak berarti untuk n besar. Kedua pengamatan ini diformalkan di Modul 06 menjadi notasi Θ, O, dan Ω. Misalnya $T(n) = an^2 + bn + c$ ditulis $T(n) = \Theta(n^2)$.

Hierarki pertumbuhan. Urutan $1 \prec \lg n \prec n \prec n \lg n \prec n^2 \prec n^3 \prec 2^n \prec n!$ (dibaca "tumbuh lebih lambat dari") akan dibuktikan secara formal di Modul 07 dengan notasi little-o, menggunakan limit

$\lim_{n\to\infty} f(n)/g(n) = 0$.

Penerapan pada algoritma lain. Pengetahuan ini dipakai untuk menilai setiap algoritma di modul selanjutnya: Merge Sort $n \lg n$ (Modul 08), Matrix-Chain-Order n^3 (Modul 11), LCS mn (Modul 12), BFS dan DFS $V + E$ (Modul 17–18), dan Bellman-Ford VE (Modul 23).

KESIMPULAN

6. Kesimpulan

- Bentuk umum running time, dari yang paling lambat tumbuh: c, $\lg n$, n, $n \lg n$, n^2, n^3, 2^n, $n!$.
- Rasio pertumbuhan $f(n_2)/f(n_1)$ mengukur berapa kali lipat waktu bertambah saat data bertambah; konstanta saling menghapus.
- Untuk kenaikan n dari 5 ke 25, rasionya berturut-turut 1, 2, 5, 10, 25, 125, 2^{20}, dan $25!/5!$.
- Logaritma berbeda basis hanya berselisih faktor konstan; sifat $\lg x^a = a \lg x$ memudahkan perhitungan rasio.
- Percepatan perangkat keras hanya memberi faktor konstan, sedangkan perbedaan bentuk fungsi memberi faktor yang terus membesar. Untuk data besar, memilih algoritma yang tepat jauh lebih efektif.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PERHITUNGAN**

Hitung rasio pertumbuhan dari $n = 8$ menjadi $n = 64$ untuk (a) $\lg n$, (b) $n \lg n$, (c) n^2.

2 **PILIHAN GANDA**

Urutan yang benar dari fungsi yang paling lambat tumbuh ke paling cepat tumbuh adalah ...

A. n, $\lg n$, $n \lg n$, 2^n, n^2
B. $\lg n$, n, $n \lg n$, n^2, 2^n
C. $\lg n$, $n \lg n$, n, n^2, $n!$
D. 1, n, $\lg n$, n^3, 2^n

3 **BENAR/SALAH DENGAN ALASAN**

"Algoritma dengan running time $1000\,n$ selalu lebih lambat daripada algoritma dengan running time n^2, karena koefisiennya jauh lebih besar." Benar atau salah?

4 **PERHITUNGAN**

Sebuah algoritma kubik ($c\,n^3$) memproses 100 data dalam 2 detik. Berapa perkiraan waktunya untuk 400 data?

5 **ANALISIS KASUS**

Algoritma A ($c_4\,n \lg n$) dan algoritma B ($c_5\,n^2$) sama-sama membutuhkan T detik untuk $n = 16$. Untuk $n = 256$: (a) hitung waktu A dan B; (b) berapa kali lebih cepat prosesor yang dibutuhkan B agar setara dengan A?

6 **PILIHAN GANDA**

Mengapa basis logaritma tidak diperhatikan ketika membicarakan orde pertumbuhan?

A. Karena semua logaritma bernilai sama.
B. Karena logaritma dengan basis berbeda hanya berselisih faktor konstan.
C. Karena komputer selalu memakai basis 10.
D. Karena fungsi logaritma tidak pernah muncul dalam analisis algoritma.

7 **ANALISIS**

Sebuah algoritma brute force berjalan dalam 2^n langkah. Komputer baru 1.024 kali lebih cepat. Jika di komputer lama dalam satu jam kita dapat menyelesaikan masalah berukuran n_0, berapa ukuran masalah yang dapat diselesaikan di komputer baru dalam satu jam?

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Perhatikan $64 = 8^2$ dan $\lg 8 = 3$, $\lg 64 = 6$.
(a) $\frac{\lg 64}{\lg 8} = \frac{6}{3} = 2$. (b) $\frac{64 \cdot 6}{8 \cdot 3} = \frac{384}{24} = 16$. (c) $\frac{64^2}{8^2} = 8^2 = 64$.

Soal 2

Jawaban: B. $\lg n \prec n \prec n \lg n \prec n^2 \prec 2^n$. Pilihan A keliru menempatkan 2^n sebelum n^2; C keliru menempatkan $n \lg n$ sebelum n; D keliru menempatkan $\lg n$ setelah n.

Soal 3

Salah. $1000n < n^2$ untuk $n > 1000$. Untuk data kecil ($n < 1000$) algoritma kuadrat memang lebih cepat, tetapi untuk $n > 1000$ algoritma linier selalu menang, dan selisihnya terus membesar. Untuk $n = 10^6$: 10^9 langkah vs 10^{12} langkah.

Soal 4

Rasio $= \frac{400^3}{100^3} = 4^3 = 64$. Perkiraan waktu $= 2 \times 64 =$ **128 detik**.

Soal 5

(a) $\text{Rasio}_A = \frac{256 \lg 256}{16 \lg 16} = \frac{256 \cdot 8}{16 \cdot 4} = 32$, sehingga A butuh $32\,T$ detik. $\text{Rasio}_B = \frac{256^2}{16^2} = 16^2 = 256$, sehingga B butuh $256\,T$ detik. (b) Prosesor B harus $\frac{256}{32} =$ **8 kali** lebih cepat.

Soal 6

Jawaban: B. ${}^a\log n = \frac{{}^b\log n}{{}^b\log a}$, dan ${}^b\log a$ adalah konstanta. Faktor konstan tidak mengubah orde pertumbuhan.

Soal 7

Di komputer baru, dalam satu jam dapat dijalankan $1024 \cdot 2^{n_0} = 2^{10} \cdot 2^{n_0} = 2^{n_0+10}$ langkah. Jadi ukuran masalah hanya naik menjadi $n_0 + 10$. Komputer seribu kali lebih cepat hanya menambah 10 pada ukuran masalah: inilah sebabnya algoritma eksponensial tidak dapat "ditolong" dengan perangkat keras.

Referensi

Sumber utama: Materi 3. PERTUMBUHAN FUNGSI RUNNING TIME; PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Kompleksitas Algoritma) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)